

A LESSON IN RESEARCH METHODOLOGY

Mathematical Modelling in Computing Science

Abstraction, Derivation, and Validation

How to reason quantitatively about a system you have not built, or cannot afford to measure — and how to establish that the reasoning is worth anything.

Research Methodology in CS · Unit II · Lecture 5



Why Build a Model at All?

A mathematical model is the bridge between Lecture 3's theoretical models and Lecture 4's measurements: it lets you reason quantitatively about a system that does not yet exist.

- 1 Models answer questions measurement cannot. What happens at ten times the load? Where would the bottleneck move under a different arrival pattern? What is the optimum, rather than merely what this configuration happened to achieve?
- 2 A model is a deliberate simplification built for a purpose. Its worth is not how faithful it is, but how useful its predictions are within a stated scope — the same fidelity-against-tractability trade Lecture 3 named for models of computation.
- 3 In computing science the thing modelled is almost always a resource under contention: queues, caches, links, schedulers, and the workloads driving them. That is why queueing and Markov models dominate the field's toolkit.
- 4 The mathematics is the easy half. The hard half is deciding what to leave out, saying so, and then establishing that what remains still predicts anything.

THE MODELLER'S BARGAIN

Every simplification buys tractability and spends fidelity. A model is honest when it says what it spent.

Which is why the question is never “is this model true?”. It is “is this model adequate for the question I am asking, over the range I am asking it in?”

Adequacy is a claim about scope — so scope must be reported.

The Anatomy of a Model

Six components. Papers routinely publish the first four and omit the last two — which are precisely the ones a reader needs in order to reuse or challenge the model.

1 Entities

The things the model contains: jobs, servers, packets, nodes, processes, users. Choosing them is already an act of abstraction.

2 State variables

What changes over time: queue length, cache contents, number in system, remaining work. The state is what the model must track to keep predicting.

3 Parameters

Quantities fixed within a run but varied across experiments: arrival rate λ , service rate μ , buffer size, number of replicas.

4 Relations

The equations, transitions, or constraints tying entities, state, and parameters together. This is the model proper — everything else is scaffolding around it.

5 Assumptions

What is asserted about the world so that the relations hold: independence, stationarity, exponentiality, no failures. These are the premises, in Lecture 3's sense.

6 Scope

The range of conditions over which the model claims to say anything at all — load levels, system sizes, workload types, time horizons.

Classes of Model

Five choices, made explicitly or by default, that determine what kind of object you have built.

THE CHOICE	ONE POLE	THE OTHER POLE	HOW TO DECIDE
Determinism	Deterministic: same input, same output.	Stochastic: randomness is part of the model; outputs are distributions.	Stochastic whenever variability is what you are studying — averages hide the tail.
Time	Static: a snapshot, with no evolution.	Dynamic: state evolves; transient and steady-state behaviour differ.	Dynamic if warm-up, bursts, or convergence are part of the question.
State space	Discrete: countable states and events.	Continuous: real-valued state, differential or fluid equations.	Discrete for modest systems; fluid approximations for very large ones.
Purpose	Descriptive: reproduces observed behaviour.	Prescriptive: recommends a decision — optimisation or control.	Say which. A descriptive model used to justify a decision is being misused.
Derivation	Mechanistic: built from known structure.	Empirical: fitted to data, with no structural claim.	Mechanistic models may extrapolate. Empirical ones usually must not.

The last row is the one most often left implicit — and the one that decides whether you are allowed to use the model outside the range you fitted it on.

PART 1 → PART 2

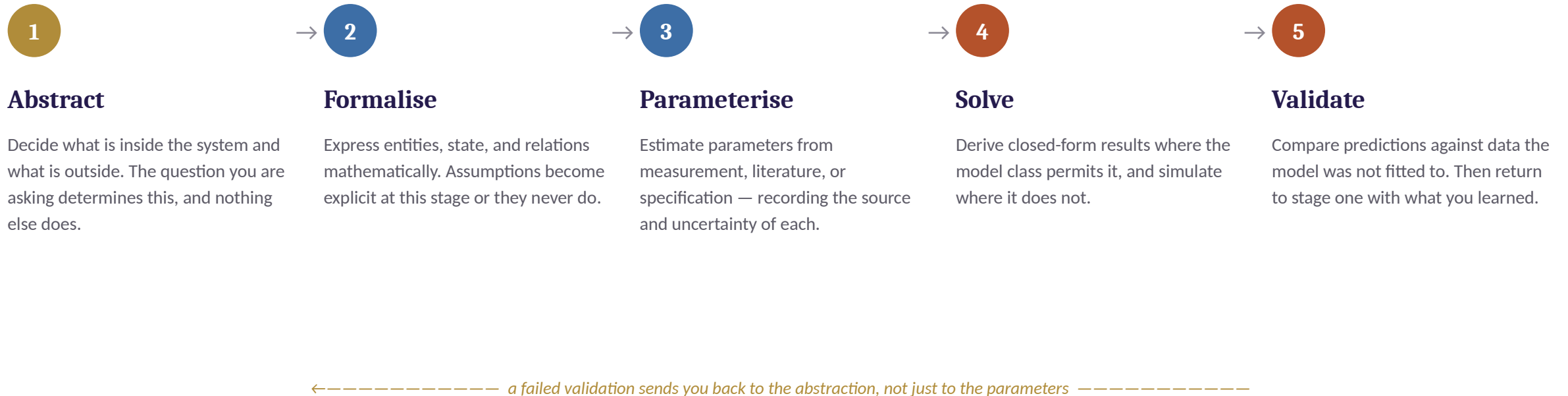
From what a model is to how one is actually built

The mathematics is rarely the difficult part. The difficult part is deciding what to leave out — and then finding out whether what remains still predicts anything.

Next: the modelling cycle, abstraction, the model-class toolkit, and a derivation carried out in full.

The Modelling Cycle

Five stages, and a loop. Compare Lecture 4's algorithm-engineering cycle: the same structure, because it is the same idea.



The loop is the point. A model that was never revised after contact with data has not been validated — it has merely been published. And note where a failure sends you: if the structure is wrong, no amount of re-fitting the parameters will repair it, though it will usually hide it.

Abstraction: Deciding What to Leave Out

The least teachable stage, and the one that decides whether everything downstream is worth doing.

THE RULE THAT GOVERNS EVERYTHING ELSE

There is no such thing as a model of a system.
There is only a model of a system, for a question.

- **“Will the queue stay stable?”**

You need λ and μ , and essentially nothing else. Service-time distribution, scheduling policy and buffer size are all irrelevant to that one question.

- **“What is the 99th-percentile latency?”**

Now the service-time distribution is the whole answer, and the mean is nearly useless. A different question has made a discarded detail essential.

- **“Will this deployment fit in memory?”**

The queueing model is irrelevant entirely. A different question calls for a different model, not a more detailed version of this one.

FOUR WORKING RULES

- 1 **Include what the question is sensitive to.**

Everything else is decoration, and decoration costs tractability without buying accuracy where it matters.

- 2 **Prefer the simplest model that still discriminates.**

If two candidate answers give the same prediction, the model cannot help you choose between them.

- 3 **Add detail only when a validation failure demands it.**

Pre-emptive complexity produces models nobody can solve, parameterise, or check. Let the data ask for each addition.

- 4 **Record every omission.**

An omission you cannot name is an assumption you cannot check — and it will be the one a reviewer finds.

The Model-Class Toolkit

Eight families that between them cover most quantitative modelling in computing science. Recognising which one a problem admits is most of the work.

Queueing models

Jobs, servers, and waiting.

*M/M/1, M/G/1, queueing networks.
Response time, utilisation, throughput.*

Markov chains

Memoryless transitions with fixed rates.

*Reliability, cache behaviour, protocol
analysis. Solved by balance equations.*

Graph models

Structure as vertices and edges.

*Networks, dependencies, citation and social
structure. Connectivity, flow, centrality.*

Optimisation models

Decision variables, objective, constraints.

*LP, ILP, convex programming. Scheduling,
placement, allocation.*

Game-theoretic models

Several agents with conflicting aims.

*Equilibria and mechanism design.
Congestion, auctions, incentive-compatible
protocols.*

Fluid and differential

Continuous approximation of large discrete systems.

*Epidemic spread, congestion-control
dynamics, large-scale load behaviour.*

Stochastic processes

Randomness evolving in time, beyond Markov.

*Arrival processes, renewal theory, heavy
tails in traffic and file sizes.*

Information-theoretic

Uncertainty and capacity as measurable quantities.

*Entropy, channel capacity, compression and
coding bounds.*

Most real modelling combines them: a queueing network whose service rates come from a measured empirical distribution, optimised over a decision variable, and checked by simulation.

Parameters: Estimation and Calibration

A model with unstated parameter provenance is not reproducible, and a model calibrated on the data used to validate it has not been validated at all.

CONCERN 1

Provenance

Where did each number actually come from?

IN PRACTICE

Measured on a running system, taken from published literature, read off a specification sheet, or estimated by hand. These are not equally trustworthy — say which for every parameter.

CONCERN 2

Identifiability

Can the data you hold determine these parameters uniquely?

IN PRACTICE

Two parameters that only ever appear as a product cannot be separated by any quantity of data. Fitting them anyway produces confident numbers that mean nothing.

CONCERN 3

Calibration $\neq$ validation

Fitting parameters so the model reproduces a dataset proves only that the fit succeeded.

IN PRACTICE

This is Lecture 3's overfitting in modelling dress. Validate on data held back from the fitting, or the exercise is circular.

CONCERN 4

Uncertainty

A parameter estimated from data has a confidence interval.

IN PRACTICE

Propagate it. Report predicted ranges rather than single numbers whenever the inputs were themselves uncertain — otherwise the output's precision is fictional.

A useful test: could a reader rebuild your model from the paper and get your numbers? If any parameter has no stated source, the answer is no.

Worked Example: Deriving the M/M/1 Queue

Question: “For a single server with random arrivals, how does mean response time depend on load?”

THE MODEL

Arrivals: Poisson, rate λ

Service: exponential, rate μ , one server

Queue: infinite capacity, first-come-first-served

Stability: $\rho = \lambda/\mu < 1$

WHAT THE ASSUMPTIONS BUY

Memorylessness is what makes the number in system a sufficient state. Without it the chain would have to remember elapsed service times, and no closed form would follow.

Read the result again: as λ approaches μ , response time diverges. The model's most useful output is not a number — it is that shape.

1

State and transitions

Let the state be n , the number in system. From state n the chain moves up at rate λ , and for $n \geq 1$ down at rate μ . This is a birth–death process.

2

Balance equations

In steady state, flow across the cut between n and $n+1$ must balance: $\lambda \cdot p_n = \mu \cdot p_{n+1}$. Hence $p_{n+1} = \rho \cdot p_n$, and so $p_n = p_0 \cdot \rho^n$.

3

Normalise

Probabilities sum to one: $p_0 \cdot \sum \rho^n = p_0 / (1 - \rho) = 1$. Therefore $p_0 = 1 - \rho$, giving $p_n = (1 - \rho) \cdot \rho^n$ — a geometric distribution.

4

Derive the answers

Mean number in system $L = \sum n \cdot p_n = \rho / (1 - \rho)$. By Little's Law, mean response time $W = L / \lambda = 1 / (\mu - \lambda)$.

And that shape is a consequence of the exponential service assumption — which real service times, with their heavy tails, routinely violate. The M/G/1 formula exists precisely because that assumption had to be relaxed.

Operational Laws: Checks That Hold Regardless

A small set of relations that follow from bookkeeping alone. They assume nothing about distributions, scheduling, or independence — which makes them the model-builder's sanity checks.

LAW 1

Little's Law

$$L = \lambda \cdot W$$

The mean number of jobs in a system equals the arrival rate times the mean time each spends there.

Holds for any arrival process, any service distribution, and any scheduling discipline — provided the system is stable and you measure L , λ and W over the same boundary.

LAW 2

Utilisation Law

$$U = X \cdot S$$

A resource's utilisation equals its throughput times its mean service time per visit.

Gives you an immediate consistency check between three quantities that simulations and measurements report separately — and often inconsistently.

LAW 3

Forced Flow Law

$$X_i = V_i \cdot X$$

Throughput at a device equals system throughput times the number of visits each job makes to it.

Turns a system-level measurement into per-device predictions, and exposes the bottleneck as the device with the largest product of visits and service time.

If your simulation output violates Little's Law, you do not have an interesting finding — you have a bug. Check the operational laws before you interpret anything else.

Analytical Solution or Simulation?

Two ways to extract predictions from the same model, with opposite strengths. The choice is a methodological decision that must be defended, not a matter of taste.

ANALYTICAL

Closed-form expressions relating inputs to outputs.

GIVES YOU

The shape of a dependence, not merely points on it. $W = 1/(\mu - \lambda)$ tells you about every load at once.

STRENGTHS

Exact, instant across the whole parameter space, and sensitivity analysis reduces to differentiation.

COSTS

Available only for restricted model classes. The closed form is usually bought with assumptions the real system violates.

USE WHEN

The question is about trends, limits, asymptotics, or optimising over a parameter.

SIMULATION

Sample paths, from which statistics are estimated.

GIVES YOU

Estimates at the parameter points you actually ran — with confidence intervals attached to every one.

STRENGTHS

Almost no restriction on structure: arbitrary distributions, priorities, failures, dependence, finite buffers.

COSTS

Every output is a statistical estimate, so all of Lecture 2's empirical obligations apply in full. Expensive.

USE WHEN

The realistic features you need destroy analytical tractability — which is most of the time.

The mature answer is usually both: an analytical model to understand the shape, a simulation to check it under realistic conditions, and measurement to validate the simulation.

Discrete-Event Simulation and Its Obligations

A clock, a future-event list, and a state. Repeatedly advance the clock to the next scheduled event, execute it, and schedule its consequences. Time moves by events, not by ticks — which is what makes it efficient.

Warm-up

Steady-state statistics must exclude the transient start-up period. Deleting too little biases toward the empty system; deleting too much wastes runs and widens intervals.

Replications

One run is one sample path, not an answer. Estimate from independent replications and report confidence intervals — never a single number.

Run length

Determined by the precision the question needs, computed in advance — not by how long you were willing to wait for the job to finish.

Random streams

Use a separate stream per stochastic component, so that changing one part of the model does not perturb the random draws in another. Report the seeds.

Common random numbers

When comparing alternatives, drive them with the same random inputs. This is variance reduction, and it makes a fair comparison dramatically cheaper.

Verification

Confirm the simulator implements the model: trace small cases by hand, and compare against an analytical special case such as M/M/1 wherever one exists.

Worked Example: Formulating an Optimisation Model

Question: “Assign a set of tasks to a set of machines so that the time until the last one finishes is as small as possible.”

THE FORMULATION

Sets: tasks $i \in T$, machines $j \in M$
 Data: p_{ij} = time for task i on machine j
 Variables: $x_{ij} \in \{0,1\}$, $C \geq 0$

minimise C

subject to $\sum_j x_{ij} = 1$ for all i
 $\sum_i p_{ij} x_{ij} \leq C$ for all j

Notice that the second constraint does all the work of the objective. Most of the skill in optimisation modelling lies in expressing what you want as something a solver will accept — without quietly changing what you meant.

1

Identify the decision

What is actually being chosen? Here it is the assignment x_{ij} . Everything else is either data or a consequence, and getting this wrong yields a precise model of the wrong problem.

2

Linearise the objective

“Minimise the maximum load” is not a linear function. Introducing C , and forcing it above every machine's load, converts it into one — a standard move worth recognising on sight.

3

State what is assumed away

No precedence between tasks, no machine failures, no setup or migration costs, and processing times known exactly in advance. Each is a modelling decision, and each is where this model can be wrong.

4

Choose a solution method

This formulation is NP-hard, so Lecture 4's four bargains apply: an exact solver on small instances, an approximation with a proved ratio, or a heuristic carrying experimental evidence.

Sensitivity Analysis

Determining how the model's outputs respond to changes in its inputs — that is, which parameters are actually driving your conclusion.

APPROACH 1

Local sensitivity

Partial derivatives, or one-at-a-time perturbation around the operating point.

IN PRACTICE

Cheap and often sufficient. Misleading whenever parameters interact, because it only ever explores one axis at a time.

APPROACH 2

Global sensitivity

Vary parameters jointly across their plausible ranges.

IN PRACTICE

Latin hypercube sampling, variance-based indices. Necessary when the model is non-linear or the parameters are coupled — which is the normal case.

APPROACH 3

Extreme-condition tests

Push parameters to their limits and see whether the model still behaves sensibly.

IN PRACTICE

Zero load, saturation, a single server, infinite capacity. A model that behaves absurdly at $p \rightarrow 1$ is telling you something structural.

APPROACH 4

Why it is not optional

It is how you find out whether you have a result or a restatement of a guess.

IN PRACTICE

If the conclusion flips when a parameter you estimated by hand moves ten per cent, then that guess — and not your model — is what produced the finding.

Report it. A sensitivity analysis showing your conclusion is robust across the plausible parameter range is often the most persuasive paragraph in a modelling paper.

Verification, Validation, Calibration

Three words used interchangeably in student write-ups, which name three entirely different questions and demand three different kinds of data.

QUESTION 1

Verification

“Did I build the model right?”

Does the implementation correctly realise the mathematics I intended? Checked by tracing small cases, unit tests, and comparison against analytical special cases.

An internal question. It never involves the real system at all.

QUESTION 2

Validation

“Did I build the right model?”

Does the model correspond adequately to the real system, for the intended purpose and over the intended range? Checked against data the model was not fitted to.

An external question — and the only one that licenses a claim about reality.

QUESTION 3

Calibration

“What parameter values make it fit?”

Tuning parameters until the model reproduces observed data. Necessary, routine, and not evidence of anything on its own.

A model can be calibrated to fit almost anything and still be structurally wrong.

The commonest modelling error in student work is presenting calibration results as validation: fitting on a dataset, then showing the fit on that same dataset as evidence the model is right.

How a Model Is Validated

Five techniques, roughly ordered by how hard they are to pass. A model reported with only the first has been checked in the weakest way available.

1

Face validity

Does it behave as domain experts expect? Weak evidence, but a cheap filter that catches gross structural errors early.

2

Extreme conditions

Set parameters to their limits — zero load, saturation, one server, infinite buffer — and check the model degenerates correctly.

3

Trace validation

Drive the model with a recorded workload trace and compare its output against what the real system actually did.

4

Predictive validation

Fit on one period or configuration, predict a different one, and compare. The strongest routine check available.

5

Structural validation

Compare the model's mechanism, not just its outputs, against known system behaviour. Two wrong assumptions can cancel and still fit.

WHY OUTPUT AGREEMENT IS NOT ENOUGH

A model can reproduce observed output through compensating errors — one mechanism overstated, another understated — and will then fail the moment conditions change. Structural checks are what distinguish a model that works from one that happens to fit.

HOW MUCH AGREEMENT IS ENOUGH?

There is no universal threshold — it depends on the decision the model supports. State the accuracy you need before you validate, so that “close enough” is a criterion rather than a verdict reached after seeing the plot.

Where Model Error Comes From

Four distinct sources. They are not equally detectable, and calibration can conceal the most serious of them.

SOURCE 1

Structural error

The relations themselves are wrong: a mechanism omitted, or a dependence assumed away.

WHY IT MATTERS

The most serious and least detectable, because fitting parameters can absorb it — producing a model that matches the data and mispredicts everything new.

SOURCE 2

Parameter error

The structure is right but the values are off.

WHY IT MATTERS

Measurement noise, a small sample, or an estimate imported from a different system. Detectable, and quantifiable through sensitivity analysis.

SOURCE 3

Numerical error

Introduced by how the model was solved rather than by the model itself.

WHY IT MATTERS

Discretisation, truncation, floating-point accumulation, or a solver stopped short of convergence. Usually small, occasionally decisive.

SOURCE 4

Scope error

The model is used outside the range it was built and validated for.

WHY IT MATTERS

This is extrapolation, and it is the commonest way a perfectly good model produces a confidently wrong answer.

Only two of the four are visible in a goodness-of-fit plot. That is why validation has to include structural and extreme-condition checks, and not just agreement on the data you happen to hold.

All Models Are Wrong

George Box's remark is quoted so often that its methodological content is usually lost. It is a statement about scope, not a licence for imprecision.

GEORGE BOX, 1976

“All models are wrong, but some are useful.”

- **Wrongness is not a defect to be repaired.**

A model that was wrong in no respect would be the system itself, and would be no easier to reason about than the system was. Simplification is the entire mechanism by which a model earns its keep.

- **Parsimony, up to a point.**

Adding parameters always improves the fit and eventually destroys the predictive power. This is Lecture 3's overfitting again, and preferring the simpler model is an inductive bias — a necessary one, not a stylistic preference.

- **The question is adequacy, not truth.**

“Is it adequate for this question, over this range, at this accuracy?” — the same fidelity-against-tractability trade Lecture 3 named for models of computation.

WHAT THE OBLIGATION ACTUALLY IS

Not to be right. To say precisely how you are wrong.

Which mechanisms you omitted, which parameters you guessed, over what range you validated, and where the predictions should not be trusted at all.

A modelling paper that supplies those four things is useful even when its numbers are later shown to be off. One that omits them is not useful even when its numbers happen to be right.

What Must Appear in the Paper

Six elements a reader needs in order to reuse, check, or challenge your model. The last column is how often each is actually supplied.

ELEMENT	WHY A READER NEEDS IT	USUALLY OMITTED?
The purpose and question	It determines whether every simplification you made was legitimate.	Often
Assumptions, listed explicitly	They are the premises; Lecture 3 showed that is where a result's content actually sits.	Very often
Parameters, with source and uncertainty	It distinguishes measured quantities from guessed ones, which readers cannot otherwise tell apart.	Very often
Solution method and its error	Analytical, numerical, or simulated — each carries a different kind of error to account for.	Sometimes
Validation evidence, and against what data	It distinguishes a validated model from one that was merely calibrated successfully.	Very often
The range validated over	It tells a reader exactly where to stop trusting the model, which is the most useful sentence you can write.	Almost always

None of these is difficult to supply. They are omitted because they are the parts that limit the claim — which is exactly why a careful reader looks for them first.

Matching Model Class to Research Question

Find the question, then read across. The last column is the assumption a reviewer will test first.

IF YOUR QUESTION IS...	MODEL CLASS	TYPICAL OUTPUT	THE ASSUMPTION MOST LIKELY TO BREAK
“How does latency grow with load?”	Queueing model	Closed-form $W(\lambda)$, or simulated percentiles	Exponential service times; independent arrivals
“What is the long-run failure probability?”	Markov / reliability	Steady-state distribution, mean time to failure	Memorylessness; constant failure rates
“What is the best allocation here?”	Optimisation (LP/ILP)	An optimal or near-optimal decision	That the objective captures what you actually want
“How does something spread in a network?”	Graph or epidemic	Reach, threshold, time to saturation	Homogeneous mixing; a static topology
“What will self-interested agents do?”	Game-theoretic	Equilibria, price of anarchy	Rationality, common knowledge, correct payoffs

Every entry in the last column is a premise you can state, test, and if necessary relax. That is what makes a modelling paper reviewable rather than merely impressive.

Modelling Failures to Watch For

Six recurring faults. Each has appeared, in a different dialect, in one of the previous four lectures.

Calibration as validation

Parameters tuned on exactly the data later used to demonstrate that the model fits. The demonstration was guaranteed to succeed before it was run.

The unstated assumption

Exponentiality, independence, or stationarity adopted for tractability, used throughout, and never mentioned again after the first page.

Extrapolation past the range

A model fitted and checked at ten to fifty per cent utilisation, then used to make confident claims about behaviour at ninety-five.

Precision without accuracy

Four significant figures reported from a model whose input parameters were estimated to within a factor of two.

The decorative model

Mathematics added to a paper that makes no prediction, is never solved, and is never checked against anything. It signals rigour without supplying it.

Fitting the model to the conclusion

Adding structure or parameters until the desired result appears — and stopping there, which guarantees the search terminated at a flattering point.

A Checklist for Your Own Model

- ✓ **State the question before the model.** A model is built for a question, never for a system. If you cannot write the question in one sentence, you cannot yet judge which simplifications are safe.
- ✓ **List every assumption where a reader will find it.** These are your premises, and Lecture 3 established that a result's real content sits in its premises rather than its derivation.
- ✓ **Give every parameter a source and an uncertainty.** Measured, cited, specified, or estimated by hand — never leave a reader unable to tell which, or unable to propagate the error.
- ✓ **Keep verification, validation, and calibration apart.** Say which data was used for each, and never use the same data for the last two. This single discipline prevents the commonest failure in the field.
- ✓ **Run a sensitivity analysis before interpreting anything.** If a parameter you guessed can flip the conclusion, that fact is the finding — and it belongs in the paper rather than in your drawer.
- ✓ **Publish the range you validated over.** And say plainly that outside it the model makes no claim. This is Lecture 2's rebuttal component, doing its job for a model.

Discussion

In pairs, ten minutes. Bring one answer back for class critique.

- 1 Take a system you know well. Write down the simplest model that would answer one specific question about it — then list what you left out, and why each omission is safe for that question.
- 2 Find a paper in your area containing a mathematical model. Which of its assumptions is doing the most work, and is it stated where a reader will find it or buried in a preamble?
- 3 The M/M/1 result says response time diverges as λ approaches μ . Real systems do not become infinitely slow — they shed load, time out, or fail. What does that tell you about the model's scope?
- 4 Take a model validated on data from one system. What exactly would you need to check before using it on a different one — and which of Lecture 3's inferences is that check?

Key Takeaways

- 1 A mathematical model is a deliberate simplification built for a question. There is no model of a system — only a model of a system, for a purpose, over a stated range.
- 2 The modelling cycle runs abstract, formalise, parameterise, solve, validate — and loops. A model never revised after contact with data has not been validated; it has only been published.
- 3 Analytical solutions give you the shape of a dependence and are bought with restrictive assumptions; simulation relaxes the assumptions and returns statistical estimates, inheriting every empirical obligation from Lecture 2.
- 4 Verification, validation, and calibration answer three different questions and need three different datasets. Presenting calibration as validation is the commonest modelling error in student work.
- 5 Every model is wrong. The obligation is to say precisely how: which mechanisms were omitted, which parameters were guessed, and over what range the predictions may be trusted at all.

Further Reading

Box, G. E. P. “Science and Statistics.” Journal of the American Statistical Association, 1976. — the source of the remark.

Kleinrock, L. Queueing Systems, Volume 1: Theory. Wiley, 1975. — the standard derivation of the results in this session.

Lazowska, E. D., Zahorjan, J., Graham, G. S. & Sevcik, K. C. Quantitative System Performance: Computer System Analysis Using Queueing Network Models. Prentice-Hall, 1984.

Denning, P. J. & Buzen, J. P. “The Operational Analysis of Queueing Network Models.” ACM Computing Surveys, 1978. — the operational laws.

Jain, R. The Art of Computer Systems Performance Analysis. Wiley, 1991. — carried over from Lecture 2.

Law, A. M. Simulation Modeling and Analysis. 5th ed., McGraw-Hill, 2015. — warm-up, replications, and run length.

Sargent, R. G. “Verification and Validation of Simulation Models.” Proc. Winter Simulation Conference, revised across editions.

Wolsey, L. A. Integer Programming. 2nd ed., Wiley, 2020. — formulation technique for the optimisation example.

Saltelli, A. et al. Global Sensitivity Analysis: The Primer. Wiley, 2008.